T4 GPU 深度解析：16GB 能跑什么模型，为什么 decode 吃满 320GB/s 带宽

用 256-bit×10Gbps 推导 T4 的 320GB/s，算清 16GB 显存能装多大模型，推导 7B INT4 约 91 token/s 的 decode 带宽上限，并对比 L4/A10 判断 2026 年 T4 还值不值得学；并以 AI Infra 视角逐行解读完整规格表，每个参数对应一条决策依据。

AI Infra 学习笔记 · 2026-10-31 · 11 min read · 2345 words



如果你在 Colab 或 Kaggle 上用过免费的 T4，多半见过这样的场景：加载一个 7B 量化模型，生成速度只有十几 token/s，等待的时间比思考的时间还长。问题通常不在模型，也不在你的代码，而在这张 2018 年发布的卡：它的显存带宽和 70W 功耗早就写好了天花板。

规格表谁都会查，但查完往往更糊涂：8.1 TFLOPS、65 TFLOPS、130 TOPS，哪个才是真实算力？datasheet 写带宽 300GB/s，产品页写 320+GB/s，听谁的？16GB 显存到底装得下多大的模型？搜索结果里几乎全是结论式回答，很少有人给出推导。

这篇是我研究 T4 规格后的完整学习笔记。读完它，你能从一张规格表自己算出三个预算：显存带宽、模型容量、decode（解码阶段）吞吐。推导过程全部保留，每一步都能拿计算器复算。

> **TL;DR**: T4 是一张"算力平平、带宽和 INT8 是亮点"的入门数据中心推理卡。FP32 只有 8.1 TFLOPS，但 INT8 达 130 TOPS（NVIDIA 官方产品页）；16GB 显存装得下 7B INT4（约 3.5GB 权重），decode（解码阶段）受 320GB/s 带宽限制，理论吞吐上限约 91 token/s。本文给出全部推导，并解释 NVIDIA 官方 datasheet（300GB/s）

> 与产品页（320+GB/s）的口径矛盾。文中附 AI Infra 视角的逐行规格表解读：每个参数如何翻译成一条 infra 决策依据。

T4 是一张什么卡：定位与关键规格

一句话定位：T4 是 2018 年随 Turing 架构发布的入门级数据中心推理卡，设计目标不是单卡极致速度，而是在 70W 功耗内高效服务更多推理任务。

项目	数值	口径说明
架构	Turing，TU104 核心，40 个 SM（流多处理器）	白皮书
CUDA Core	2560	产品页
Tensor Core	320	产品页
显存	16GB GDDR6，256-bit 总线	产品页
显存带宽	320+ GB/s（产品页）/ 300 GB/s（datasheet）	下文带宽一节裁决
FP32	8.1 TFLOPS	产品页
FP16 Tensor Core	65 TFLOPS	产品页
INT8	130 TOPS	产品页
INT4	260 TOPS	产品页
TDP	70W	产品页

上表数值来自 NVIDIA T4 官方产品页 与 Turing 架构白皮书。有两点值得先记下。其一，70W 低于 PCIe x16 插槽 75W 的供电上限，所以 T4 不需要外接电源线（这是我的推断，官方未直接写明，但功率数字支持这个结论）。其二，带宽一栏官方自己就有两个说法，下文带宽一节会给出来龙去脉。

为什么不能只看 TFLOPS：三个数字，三种电路

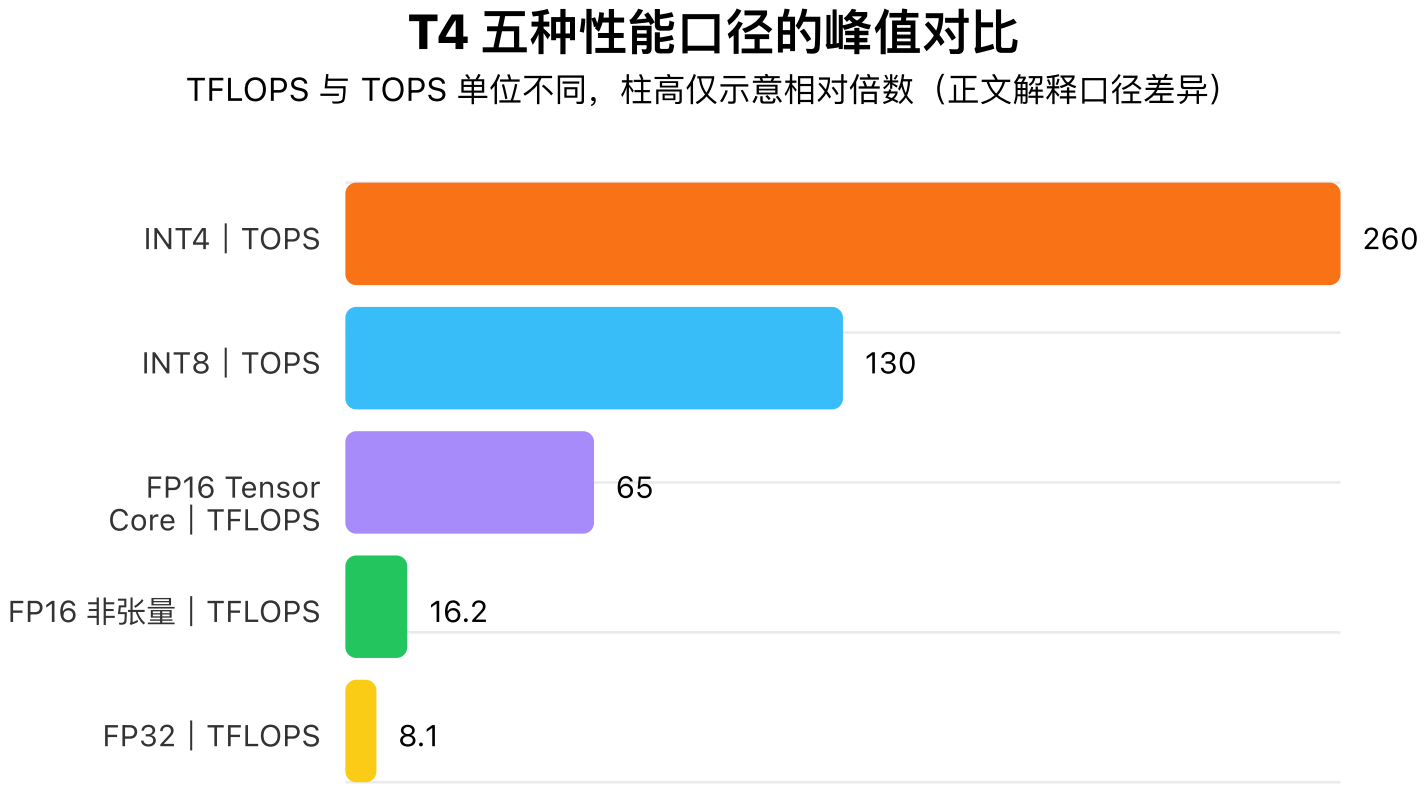
8.1 TFLOPS（FP32）、65 TFLOPS（FP16 Tensor Core）、130 TOPS（INT8）不是同一份算力的三种写法，而是三种不同计算电路各自的峰值。拿哪个数字做预算，取决于你的算子走哪条电路。

8.1 TFLOPS 是 2560 个 CUDA Core 的标量浮点峰值，走这条路的是普通算术和 elementwise 运算。如果 FP16 运算不走 Tensor Core，峰值只有 16.2 TFLOPS（白皮书口径），只是 FP32 的两倍，不是八倍。65 TFLOPS 才是 Tensor Core 矩阵路径的 FP16 峰值：

```
65 / 8.1 ≈ 8.02 倍
```

白话解释：同样标着 FP16，走没走 Tensor Core 差了 4 倍；和 FP32 比，差了 8 倍。再看整数路径：INT8 是 130 TOPS，INT4 是 260 TOPS，单位已经是 TOPS（每秒万亿次整数操作）而不是 TFLOPS（每秒万亿次浮点操作），所以柱状图上的高度不能当成同等工作量来比较。

还要泼一盆冷水：峰值不等于实际推理性能，算子能否用上 Tensor Core、batch 大小、访存效率都会让实际数字远低于标称。这套"口径决定认知"的逻辑后面还会出现两次：L4 的稀疏对密集，Orin 的 TOPS 对 TFLOPS（见后文三卡对比一节）。



Source: NVIDIA T4 官方产品页 / Turing 架构白皮书, 2018.

CUDA Core 与 Tensor Core：SM 里的分工

CUDA Core 是通用标量单元，什么都能算但单项不快；Tensor Core 是专用的混合精度矩阵乘累加单元，Turing 这一代支持 FP16 输入与 INT8/INT4 输入，累加走 FP32/INT32（白皮书口径）。LLM 推理的吞吐主要看后者。

T4 一共 40 个 SM（流多处理器），每个 SM 里 64 个 CUDA Core 和 8 个 Tensor Core：

```
CUDA Core 总数   = 40 SM × 64 = 2560
Tensor Core 总数 = 40 SM × 8  = 320
```

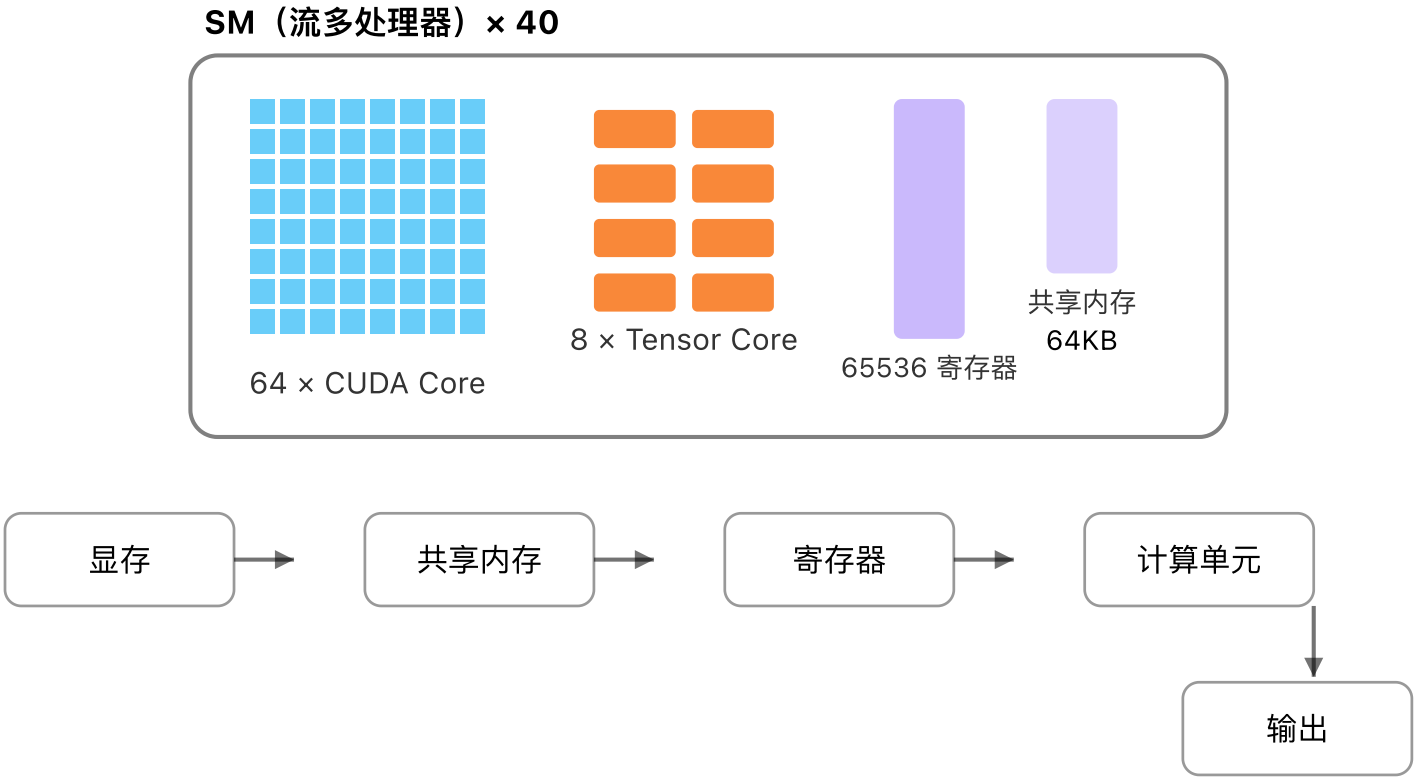
白话解释：把 SM 想成一个车间，CUDA Core 是通用机床，什么零件都能加工但速度有限；Tensor Core 是专门做矩阵乘的自动生产线，只干一件事，但快得多。

为什么 GEMM（通用矩阵乘）是 LLM 的心脏？因为 Transformer 里的 Linear、MLP、Attention 投影底层全是矩阵乘法。CUDA Core 和 Tensor Core 也不是替代关系，而是融合量化 GEMM 算子里的流水线分工：CUDA Core 做地址计算和索引，Load 指令把数据从显存搬进寄存器或共享内存，Tensor Core 完成矩阵乘，CUDA Core 再做 bias、激活、scale 后处理。理解这个分工，是研究 CUTLASS 和 TensorRT plugin 的硬件基础。

顺便纠正三个我一开始记错的细节。第一，65 TFLOPS 是 FP16 Tensor Core 的峰值口径，不是任何 FP16 算子都能摸到，不走 Tensor 路径就只有 16.2。第二，Turing（计算能力 7.5）的 Max blocks per SM 通常是 16，不是我最初抄来的 32（官方依据见下文规格表逐行解读一节）。第三，单 block 默认共享内存上限 48KB，但 Turing 每个 SM 的共享内存可配置到 64KB，用到更高需要显式 opt-in。这些限制直接决定 occupancy：每个 warp（线程束）32 个线程，每 SM 最多驻留 1024 个线程，寄存器共 65536 个 32-bit 槽位。假设一个 block 256 线程、每线程 64 个寄存器：

```
每 block 寄存器需求 = 256 线程 × 64 = 16384
可驻留 block 数     = 65536 / 16384 = 4
```

白话解释：寄存器用得越凶，能同时驻留的 block 越少，GPU 用来隐藏访存延迟的"兵力"就不足。



数据路径：显存 → 共享内存 → 寄存器 → CUDA Core / Tensor Core → 输出（示意）

Turing SM 结构简化示意。来源：NVIDIA Turing 架构白皮书，2018（本文自绘简化版）。

320GB/s 是怎么算出来的：兼解官方口径矛盾

显存带宽可以自己算：总线宽度除以 8 变成字节，再乘以有效数据速率。T4 是 256-bit 总线配 10Gbps 速率，乘出来正好 320GB/s；datasheet 的 300 是保守写法，白皮书按 10Gbps 推导就是 320。

```
BW = (256 bit / 8) × 10 Gbps
   = 32 Byte × 10 G 次/秒
```

```
= 320 GB/s
```

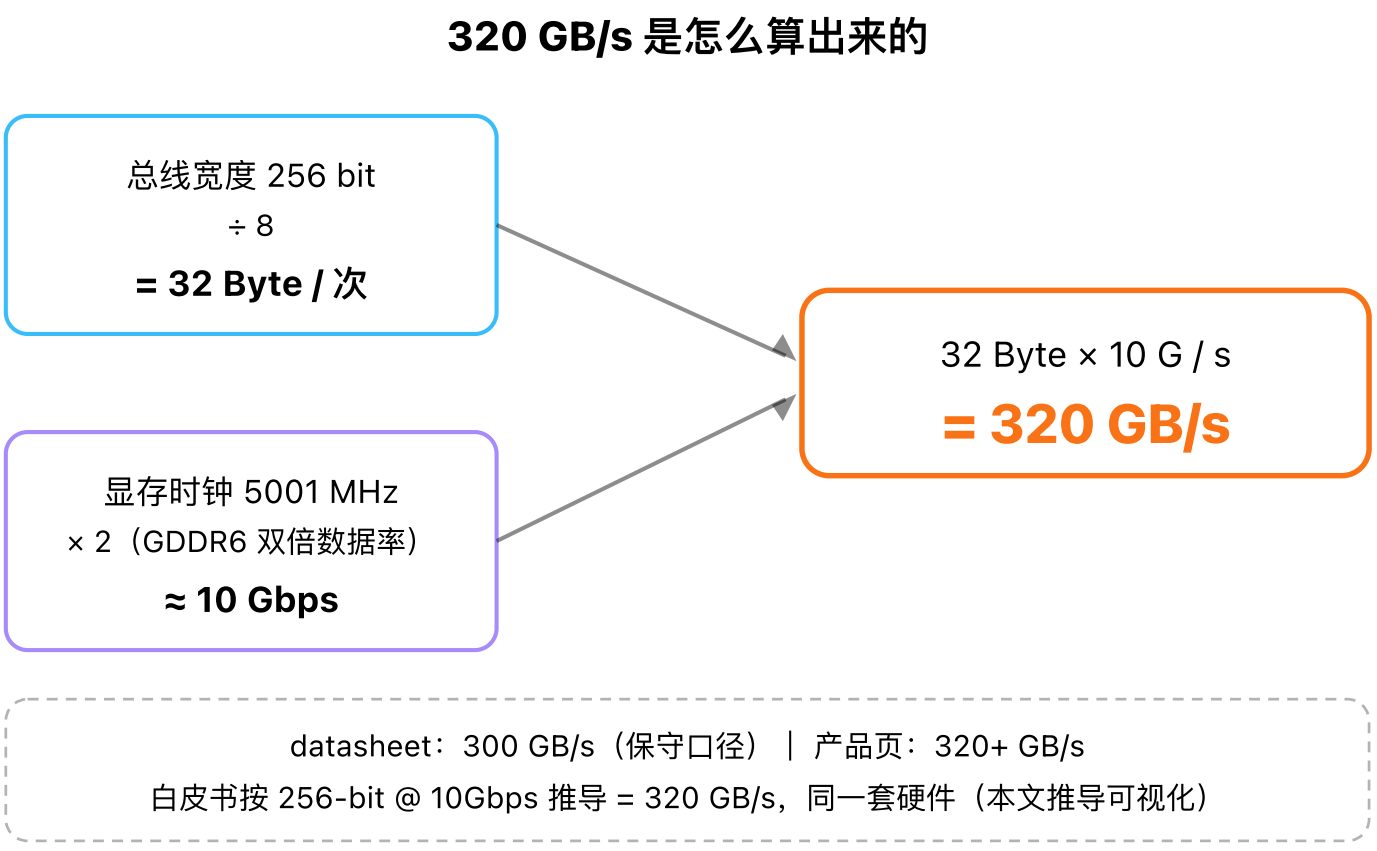
白话解释（本文推导）：显存总线有 256 根数据线，一次传 256 bit，即 32 字节；每秒传输 10G 次，32 Byte × 10G/s，正好等于 320 GB/s。

这里最容易踩坑的是规格表里的"显存时钟 5001MHz"。它不是"每秒 5001M 次传输"。GDDR6 是双倍数据率内存，时钟的上升沿和下降沿各传一次，有效速率约等于物理时钟的两倍：

```
有效数据速率 ≈ 5001 MHz × 2 ≈ 10 Gbps
```

白话解释：把 5001MHz 直接当传输速率，会算出 160GB/s 的腰斩答案；先翻倍，再进公式。

接着说官方口径矛盾。NVIDIA 的 T4 datasheet（2018 年 12 月）写 300GB/s，产品页写 320+GB/s，而白皮书按 256-bit @ 10Gbps 给出 320GB/s。我的处理方式：引用时写 320GB/s 并注明"白皮书与产品页口径"，看到 300 不要慌，那是 datasheet 的保守写法，两个数字描述的是同一套硬件。



推导口径：NVIDIA Turing 架构白皮书（256-bit @ 10Gbps），本文自绘。

16GB 能装多大模型：显存算术

FP16 最多装 7B 且几乎没有余量，INT8 7B 有运行空间，INT4 7B 是舒适区，14B 只有 INT4 能装下权重且要精打细算。方法只有一句话：参数量乘以每个参数的字节数。

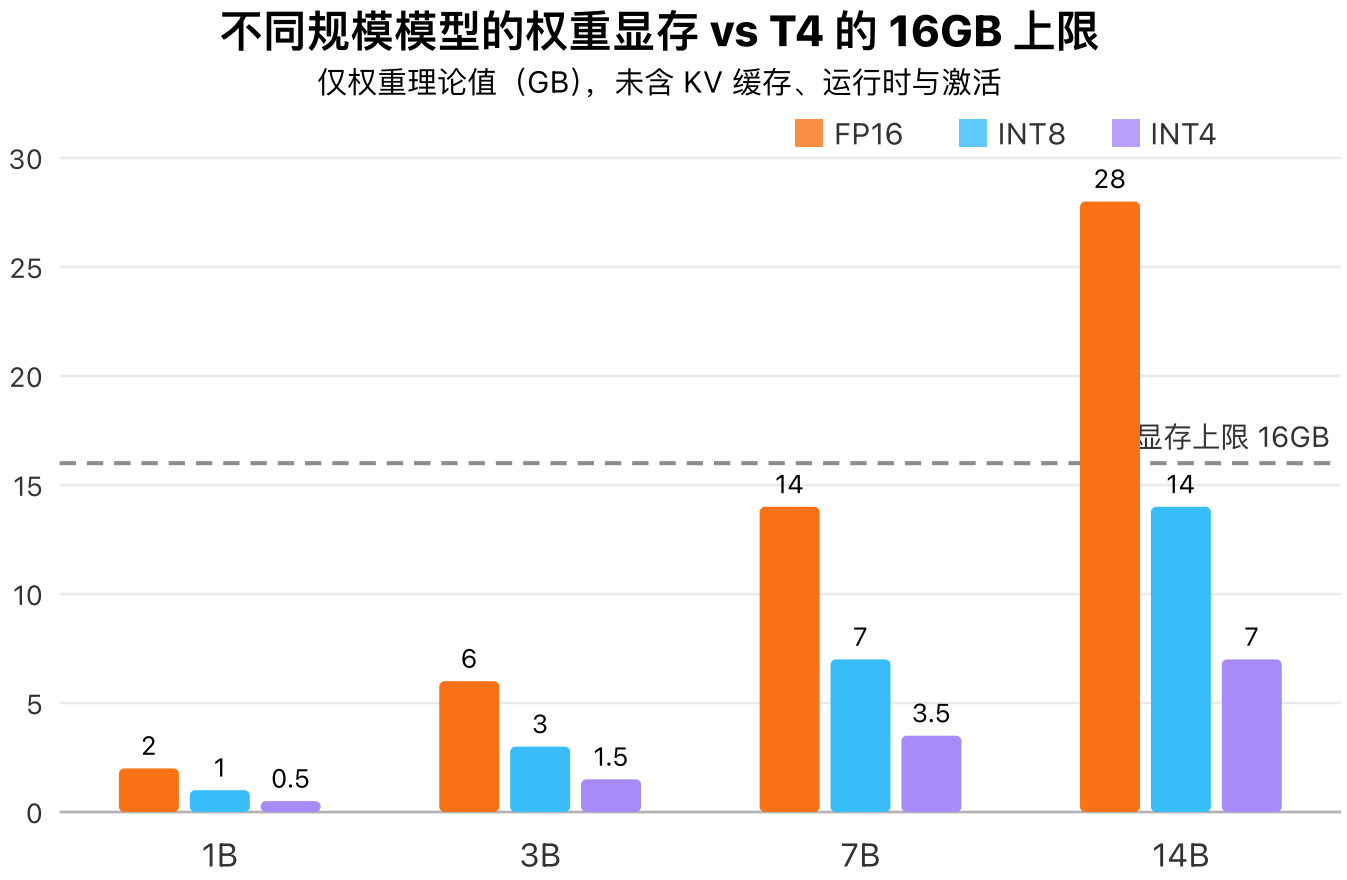
```
权重显存 ≈ 参数量 × 每参数字节数
```

```
7B × 2 Byte   (FP16) = 14 GB
7B × 1 Byte   (INT8) = 7 GB
7B × 0.5 Byte (INT4) = 3.5 GB
```

白话解释：一个参数存成什么精度，就占多少字节；模型多大、用什么精度存，显存账单当场就能算出来。扩展到常见规模（单位 GB，仅权重理论值，本文推导）：

模型规模	FP32	FP16	INT8	INT4
1B	4	2	1	0.5
3B	12	6	3	1.5
7B	28	14	7	3.5
8B	32	16	8	4
14B	56	28	14	7

但这张表只是"理论列"，现实列至少还要加三笔账。第一，INT4 不是干净的 0.5 Byte 每参数，还要存 scale、zero point 和对齐元数据，实际权重略高于表值。第二，KV 缓存是第二个大头，随上下文长度和并发请求数线性增长，长对话场景下可能反超权重。第三，CUDA context、框架运行时和激活值还要再占一块固定开销。所以"T4 能跑的最强模型"没有单一答案：单看权重，14B INT4 是上限；算上 KV 缓存和运行时，日常舒适区是 7B INT4。



14B FP16（28GB）与 FP32 各柱超出 16GB 上限，装不进 T4。

Source: 本文推导（参数量 × 精度字节数）（2026）

Source: 本文推导（参数量 × 精度字节数，方法见上文，可复算），2026-10-31。

decode 为什么吃带宽：91 token/s 是怎么推出来的

自回归生成的 decode（解码阶段）每产出一个 token 都要把全部权重读一遍，所以单请求吞吐的上限由显存带宽决定：7B INT4 权重 3.5GB，除以 320GB/s，每个 token 约 10.94ms，倒过来就是约 91 token/s。prefill（预填充阶段）恰好相反，是算力瓶颈。

```
W = 7B 参数 × 0.5 Byte (INT4) = 3.5 GB

T_weight = 3.5 GB ÷ 320 GB/s = 0.01094 s ≈ 10.94 ms / token

TPS_上限 = 1 ÷ 0.01094 ≈ 91 token/s
```

白话解释（本文推导，理想化估算）：decode 就像开卷考试，每答一道题都要把整本教材（权重）从头翻一遍，翻书的速度（显存带宽）决定了每分钟能答几题，跟写字多快（算力）关系不大。

把算力和带宽画进同一张图，就是 roofline 模型。横轴是计算强度 AI，意思是每读 1 字节显存要配多少次计算：

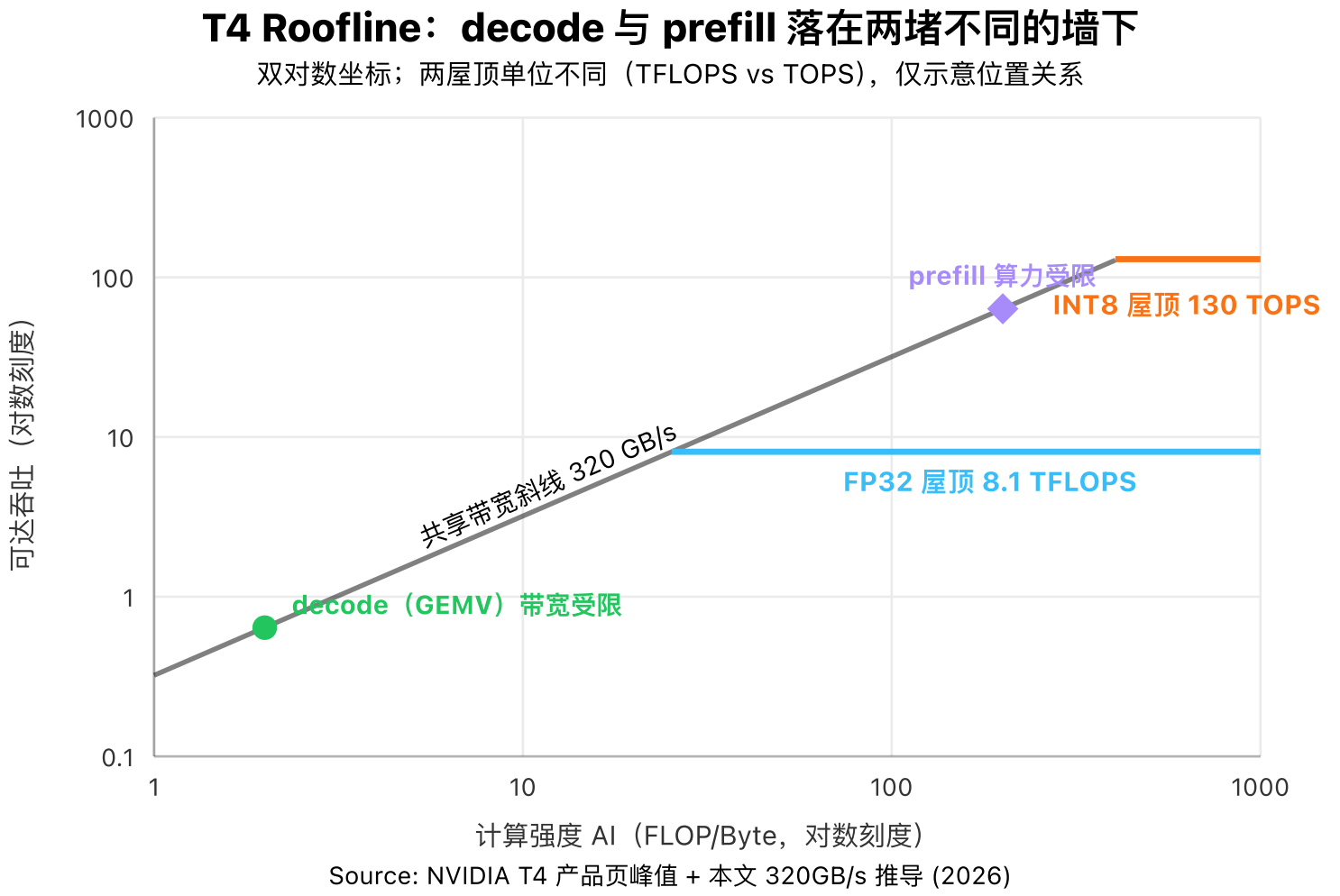
```
AI = FLOPs / Bytes

AI_FP32        = 8.1e12 ÷ 320e9 ≈ 25.3 FLOP/Byte
AI_FP16_Tensor = 65e12  ÷ 320e9 ≈ 203.1 FLOP/Byte
```

白话解释：一个算子的计算强度低于拐点，就落在带宽墙下，再高的峰值算力也是摆设；高于拐点，才轮到算力说话。decode 的 GEMV 计算强度远低于 25.3，是带宽受限；prefill 大批量并行、矩阵规整，容易吃满 Tensor Core，是算力受限。一张卡、两个瓶颈，这就是推理引擎要区别对待两个阶段的根本原因。

91 token/s 是理想上限，现实至少有八项折扣：反量化开销、算子启动开销、KV 缓存访问、非连续访存、GEMV/GEMM 利用率、batch size 影响、算子调度间隙、有效带宽低于标称值。层层叠下来，实际数字会明显低于 91。

那为什么 Colab/Kaggle 的免费 T4 连这个数都跑不到？我的理解是三重额外折扣：70W 功耗墙让 GPU 无法长时间顶在标称频率；T4 是被动散热，依赖服务器风道，云平台条件未必理想；免费平台普遍还会限频控制整机成本。推导给出理论天花板，平台条件决定你在天花板下面的哪一层。



Source: NVIDIA T4 官方产品页（峰值）+ 本文带宽推导（320GB/s），2026-10-31。拐点：FP32 约 25.3、INT8 约 406 FLOP/Byte（本文推导）。

AI Infra 视角：逐行读完规格表，每个参数对应一条决策依据

前面几节是三笔账的深挖。这一节把方法反过来用：假设你刚接手一批 T4，手里只有一张完整规格表，AI Infra 工程师的工作不是背数字，而是把每一行翻译成一条决策依据。我的读法是把全表拆成五张预算表：架构代次行决定软件生态边界，算力行决定 kernel 走哪条电路，显存与缓存行决定容量和吞吐，SM 资源行决定 kernel 调优空间，互连与功耗行决定机型和部署密度。深挖过的参数给结论和回链，前面没展开的这里补齐。

架构代次行：软件生态的边界条件

参数	T4 规格	infra 决策依据
Architecture	Turing	特性集代名词：这一代没有 BF16、TF32、结构化稀疏，也没有 MIG
Compute Capability	7.5	编译目标 sm_75，判断 kernel 库支持矩阵的第一道筛子
Manufacturing Process	12 nm FFN	同代频率与功耗的物理解释，跨代对比时用
Release Year	2018	折旧与生态维护期的粗估输入

白话解释：看到 Turing 和 7.5，有经验的工程师脑内自动展开一张"这卡没有什么"的清单——BF16、TF32、2:4 结构化稀疏、cp.async 异步拷贝都是 Ampere（CC 8.0）及以后的特性，MIG 从 A100 开始才有。对 infra 的直接含义：从 A100/H100 生态迁来的 BF16 权重和按新特性写的 kernel，到 T4 上要么转 FP16 重新校准，要么直接编译不过。所以选型除了"算力够不够"，永远要加一条"特性集够不够"；年份和制程则进成本账——2018 年的卡在 2026 年还值不值得用，先看它缺的特性能否用软件路径绕过去（INT8 就是 T4 的绕行通道）。

算力行：prefill 吞吐预算

参数	T4 规格	infra 决策依据
CUDA Cores	2560	标量路径算力的分子
SM Count	40	任务划分的颗粒度，40 个"车间"
Tensor Cores	320	矩阵路径算力的分子（40 SM × 8）
Clock Speed	585 MHz base / 1590 MHz boost	标称算力按 boost 口径算，长时负载稳不住
FP32 Performance	8.1 TFLOPS	= 2560 × 2 × 1.59 GHz，非张量算子的预算
Tensor Performance	65 TFLOPS (FP16)	量化档位选择：FP16/INT8/INT4 = 65/130/260

峰值算力本身就该手算验证一遍（本文推导）：

```
FP32 峰值 = 2560 核 × 2 FLOP/周期 × 1.59 GHz ≈ 8.1 TFLOPS
```

白话解释：官方标称用的是 boost 频率，而 70W 功耗墙下长时间推理大概率稳不住 1590 MHz——这就是"预算按标称、实际打折旧"的根源（免费 T4 为什么慢的三重折扣，见上文 decode 一节）。算力行最终汇成一个优化决策：prefill 吃 Tensor Core，量化档位每升一级峰值翻倍；decode 却只吃带宽，不吃这份红利（roofline 推导见上文）。所以"T4 上量化先救 prefill"是动手优化前的第一判断。

显存与缓存行：容量与带宽预算

参数	T4 规格	infra 决策依据
Memory	16 GB GDDR6	模型规模 × 精度的容量账（见上文）
Memory Bus Width	256-bit	带宽公式的一半：÷8 变 32 Byte/次
Maximum Memory Clock	5001 MHz	DDR 翻倍后 ≈ 10 Gbps，直接当速率会算出腰斩的 160GB/s（见上文）
Peak Memory Bandwidth	320 GB/s	decode 吞吐天花板：7B INT4 ≈ 91 token/s（见上文）

参数	T4 规格	infra 决策依据
L2 Cache Size	4 MB（TU104 架构口径，T4 官方页未单列）	decode 流式读权重基本无效，batch 内复用才受益

L2 是规格表里最容易被高估的一行。decode 每个 token 都要把 3.5GB 权重从头读一遍，4MB 的 L2 相对权重是九牛一毛，而且这些数据在"下一次读取"之前早就被挤出去了——真正能吃到 L2 的是同一批次内的数据复用：多请求共享的 system prompt KV、融合算子反复访问的激活值。白话解释：小 batch 场景别指望 cache 把带宽墙变矮，容量规划按零缓存收益做最坏预算。

SM 资源行：kernel 调优的硬约束

参数	T4 规格	infra 决策依据
Shared Memory per SM	64 KB	tile（分块）数据的驻地，直接限 tile 上限
Max Shared Memory per Block	48 KB（可 opt-in 到 64 KB）	tile 尺寸的硬上限
Registers per SM	65536 × 32-bit	occupancy 公式的分母
Max Registers per Thread	255	编译期就要守的每线程上限
Warp Size	32	调度最小单位，不足 32 线程也占一整个 warp
Max Threads per Block	1024	启动配置上限
Max Threads per SM	1024	= 32 warps/SM，驻留线程上限
Max Blocks per SM	16	小 block 多驻留策略的上限

先纠一个口径：最后一行在一些聚合网站上写着 32，那是 Volta（CC 7.0）的数字；按 NVIDIA CUDA C Programming Guide 的 compute capability 规格表，7.5 是 16。聚合站的表永远要回官方文档核对一遍——这和上文带宽的 300/320 口径矛盾是同一种病。

这组数字合起来就是 occupancy（占用率）三公式：寄存器、共享内存、线程与块数各算一遍能驻留多少 block，取最小值（演算见上文 SM 一节）。它们直接决定 kernel 写法的三个选择：

- **block 开多大**：1024 线程的 block 顶满 SM 上限，一个 block 就吃光驻留名额。
- **tile 开多大**：128×128 的 FP16 分块占 32 KB，48 KB 预算里放不下两份，要么缩 tile，要么 opt-in 到 64 KB。
- **寄存器敢用多少**：每线程 64 个寄存器时 1024 线程刚好用满 65536 个槽位，再往上就得牺牲驻留数——`__launch_bounds__` 和 `-maxrregcount` 就是在这笔账上做交易。

白话解释：Nsight Compute 报 occupancy 上不去时，答案几乎总在这四行规格里；它是手写算子、读 CUTLASS、写 TensorRT plugin 时真正的语法。

互连与功耗行：机型与部署密度

参数	T4 规格	infra 决策依据
Interconnect	PCIe Gen3 x16	单向约 16 GB/s：权重加载与 CPU offload 的过路费
TDP	70 W	低于 75 W 插槽供电，可高密度部署（免外接电源为推断，见开头定位一节）
散热（规格表不写）	被动散热	依赖服务器风道，上架检查项

PCIe 的 16 GB/s 主要影响两件事。一是模型加载：16 GB 权重理论上一秒出头就能过完 PCIe，实际加载慢通常是存储和反序列化——先量再猜，别急着怪接口。二是 CPU offload 方案：每 token 需要过 PCIe 的字节直接进延迟，7B INT4 的 3.5GB 权重若整体放在 CPU，单搬运就是 3.5 ÷ 16 ≈ 220 ms/token 的量级，GPU 上 10.94 ms 的 GEMV 预算完全被淹没——这就是"能跑起来"和"能提供服务"的差距（本文推导）。70W 则是部署侧的红利：不超过 PCIe 插槽 75W 供电上限，免外接电源线，2U 服务器插 8 张以上不稀奇；代价是功耗墙加被动散热，机箱风道不达标就是稳定性事故（回链免费 T4 慢的三重折扣）。

一句话总结：规格表不是宣传页，是预算表——算力行给 prefill 立预算，显存行给容量和 decode 立预算，SM 资源行给 kernel 立预算，架构行和互连行圈定生态与部署的边界。换一张新卡（L4、A10 还是更新的 Blackwell），这套逐行翻译的流程原样可用。

2026 年还值得学 T4 吗：与 L4、A10 和 Orin 对比

作为生产卡，T4 的位置已经被 L4 和 A10 取代；但作为"学习推理硬件的第一张卡"，它仍是我心目中性价比最高的教具之一，因为它的规格简单到可以手算，而手算能力正是推理优化的第一课。

指标	T4	L4	A10
架构	Turing（2018）	Ada	Ampere
显存	16GB GDDR6	24GB	24GB
显存带宽	320 GB/s	300 GB/s	600 GB/s
INT8 峰值	130 TOPS（密集）	485 TOPS（稀疏）/ 242.5（密集）	250 TOPS（密集）
功耗	70W	72W	150W

数据来自 NVIDIA L4 官方产品页 与 A10 官方产品页。口径陷阱又来了：L4 宣传页最显眼的 485 TOPS 是稀疏值，密集口径只有 242.5；A10 的 250 与 T4 的 130 都是密集值，同口径可比，对 485 则不可比。稀疏和密集混着比，会得出 L4 比 A10 强的错觉。L4 官方页还给出相对 T4 约 2.5 倍的 AI 图像生成性能提升，引用时要连同场景口径一起写。

软件支持边界：经典 TensorRT 对 T4 一直有完整支持，是主流部署路径；TensorRT-LLM 面向更新的架构，对 Turing 的支持情况需以官方支持矩阵为准（方向性判断：不要假设 T4 能用上其最新特性）。所谓"TensorRT 不支持 T4"是以

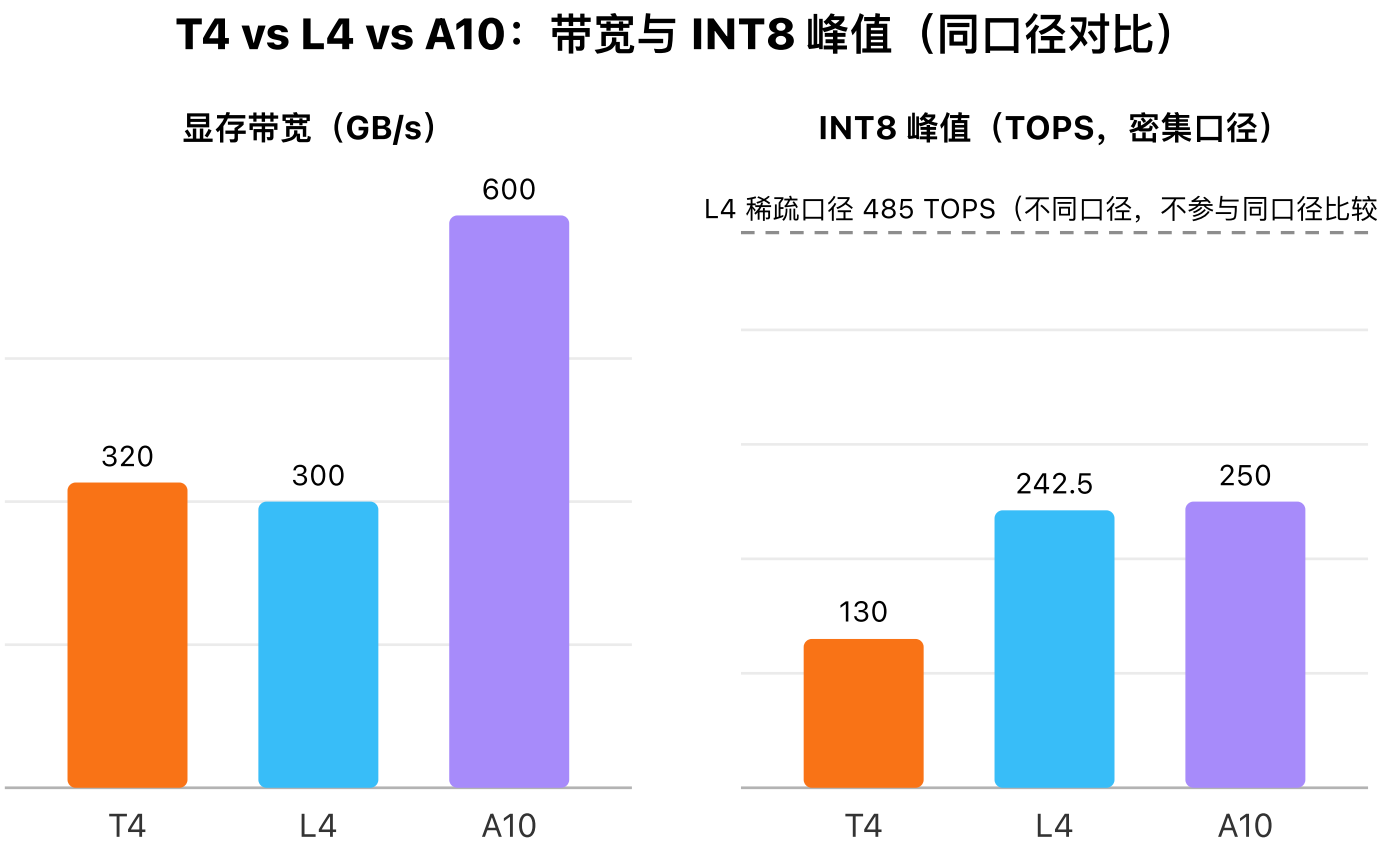
讹传讹，经典版本在 T4 上至今可复现。

再把另一张常被拿来和 T4 混谈的卡放进来：Jetson Orin Nano Super。

指标	Tesla T4	Jetson Orin Nano Super
架构	Turing	Ampere
CUDA Core	2560	1024
Tensor Core	320	32
显存	16GB GDDR6，320GB/s	8GB LPDDR5（与 CPU 共享），102.4GB/s
功耗	70W	约 25W
形态	PCIe 数据中心加速卡	边缘 SoC

最后一个口径陷阱：Orin Nano Super 宣传的 67 TOPS 是 INT8 口径，T4 的 65 TFLOPS 是 FP16 Tensor 口径，数字看着只差 2，单位完全不同，不能直接比。

我手上正好有 Orin Nano Super 的部署经验，这张表对我不只是纸面对比：T4 教你数据中心卡的经典玩法（TensorRT、INT8 校准、多租户 serving），Orin 教你在功耗墙与共享内存约束下做边缘部署。两张卡是学习路径的两条腿，跨架构对比正好训练"先问口径、再问架构、最后问数字"的习惯。



稀疏与密集混比会得出"L4 比 A10 强"的错觉；本图右面板统一为密集口径。

Source: NVIDIA T4 / L4 / A10 官方产品页 (2018-2026)

Source: NVIDIA T4、L4、A10 官方产品页。

我会用 T4 学什么：一条可复制的闭环

不要只拿 T4 跑一次 YOLO 或者部署一个 LLM 就完事。把它当成一台 AI Infra 实验设备，走完这条闭环：PyTorch 模型，ONNX 导出，量化，自定义算子，TensorRT 引擎，GPU Serving，性能 Profiling。这条链路把部署、量化、CUDA 优化、推理引擎、服务调度和性能分析全部串起来。

这条路线值得走的证据来自官方教程。GCP 的 T4 推理教程在同一张卡上做 ResNet 优化：原生 TensorFlow 314 fps，转 FP16 后 988 fps，TensorRT INT8 后 1524 fps，三步走完接近 4.9 倍提升，说明在 T4 上"INT8 值得做"不是口号。顺带一个口径提醒：教程原文同时写"延迟降至 0.08 ms"，但按 1524 fps 折算单张约 0.66 ms，两个数字对不上；0.08 s 恰好等于 batch 128 的整批耗时（128 ÷ 1524 ≈ 0.084 s），原文大概率把"秒"标成了"毫秒"（我的推断）。引用延迟数字前，先拿吞吐倒推一遍，这是防止口径陷阱的最后一道工序。LLM 侧还有旁证：Microsoft 开源博客（2022-05-02）报告 T4 上 BERT-Large INT8 延迟低至 1.2ms，注意这是 Bing/Azure、TensorRT 8 语境的数据，别和同文里 A100 的数字混用。

闭环里的每个环节都值得单独深挖：roofline 用 Nsight 实测、手写 INT4 weight-only GEMM、KV 缓存管理、连续批处理（continuous batching）、多租户 GPU serving。本文先把地基打牢。

常见问题

T4 相当于什么级别的显卡?

T4 与 GeForce RTX 2080 同为 TU104 核心，同属 2018 年的 Turing 架构（RTX 20 系内各型号核心不同：2080 Ti 用 TU102，2070/2060 用 TU106）。但它是一张数据中心形态的卡：无显示输出、被动散热、显存带 ECC，定位是服务器推理而不是桌面使用。把它当"老款游戏卡"看待，会误解它的设计目标。

为什么 Colab/Kaggle 的免费 T4 跑大模型很慢?

先撞带宽墙：7B INT4 的 decode 理论上限约 91 token/s（推导见上文），这是 320GB/s 显存带宽决定的物理天花板，再叠加反量化、KV 缓存访问等现实折扣。云平台还有 70W 功耗墙、被动散热和限频三重折扣，实测十几 token/s 属于正常现象，不是环境坏了。

T4 和 P40 做推理选哪个?

追求 INT8/INT4 吞吐选 T4：它有完整的 Tensor Core 整数路径（130 TOPS 口径）。P40 没有 Tensor Core，INT8 靠 CUDA Core 的 DP4A 指令实现，量化收益有限；它的讨论价值主要在更大的显存装更大的 FP16 权重。老卡选型的关键动作，就是先问"我的算子走哪条电路"。

如何在 T4 上用 TensorRT 做 INT8?

按上文 GCP 官方教程走即可复现：模型导出 ONNX，TensorRT 构建引擎，执行 INT8 校准，教程里 ResNet 从 314 fps 提升到 1524 fps。记住口径提醒：130 TOPS 是 Tensor Core 峰值，实际收益取决于算子覆盖率和校准质量，不是标称值的固定折扣。

T4 能跑 14B 模型吗?

能装下，但仅限 INT4：14B × 0.5 Byte = 7GB 权重（推导见上文容量算术），16GB 显存还剩 9GB 给 KV 缓存、运行时和激活值。上下文一长、并发一多就会触顶，所以"能装下权重"和"能稳定服务"是两回事，上车前先做一遍显存预算。

结论

T4 教给推理工程师的是三笔账和两个警觉：带宽账（256-bit 与 10Gbps 推出 320GB/s）、容量账（参数量乘精度字节数）、吞吐账（3.5GB 除以带宽得到约 91 token/s）；警觉口径陷阱（稀疏对密集、TOPS 对 TFLOPS）和峰值幻觉（65 TFLOPS 是 Tensor 路径峰值）。这些手算能力比任何单卡结论都长寿。文中每个推导都给了完整算式，拿计算器就能复算与勘误。

2026-10-31